MÁSTER FP · INTELIGENCIA ARTIFICIAL Y BIG DATA

Desarrollo guiado por especificación con Cursor

Spec-Driven Development aplicado a Visor Pokémon

Sergii Butrii · Cristian Sánchez · Daniel Meléndez

¿QUÉ ES SDD?

La especificación manda. El código la sirve.

Primero se escribe qué hay que construir. Después la IA lo implementa. Una persona dirige y supervisa.

El repositorio tiene dos capas

1 · QUÉ ES LA APLICACIÓN

specs/requisitos.md
qué debe hacer: R1 a R14

specs/diseno.md
stack, rutas, datos y aspecto

specs/tareas.md
orden de trabajo: T1 a T7

Markdown normal: Cursor no impone formato

2 · CÓMO SE COMPORTA LA IA

AGENTS.md
el contrato, en todas las sesiones

.cursor/rules · skills · agents · commands
normas, procedimiento, subagentes y atajos

.cursorignore
lo que el agente no debe mirar

Formatos propios de Cursor

Fases del proceso

#	Fase	Dónde	Qué se hace
1	Requisitos	requisitos.md	Alcance y criterios comprobables
2	Diseño	diseno.md	Pantallas, datos, archivos y aspecto
3	Tareas	tareas.md	Pasos pequeños, en orden
4	Implementación	/implementar-tarea	La IA programa una sola tarea
5	Revisión	revisor-spec	Cumple o no cumple, criterio a criterio

Cambio de requisitos: se vuelve a la fase 1. Primero la spec, después el código.

Las piezas de Cursor

Pieza	Dónde vive	Para qué sirve	En el proyecto
AGENTS.md	raíz	Contrato presente en todas las sesiones	Sí
Rules	.cursor/rules/*.mdc	Normas cortas según el contexto	3
Skills	.cursor/skills/	Un procedimiento paso a paso	1
Subagentes	.cursor/agents/	Agentes con su propio contexto	2
Commands	.cursor/commands/	Atajos que se lanzan con /	1
Hooks	.cursor/hooks.json	Scripts en momentos del ciclo	No
MCP	.cursor/mcp.json	Conectar herramientas externas	No

Las piezas son de Cursor. Cómo combinarlas lo decide cada equipo.

Ejemplo: implementar una tarea

PERSONA

`/implementar-tarea T3`

Elige qué tarea se hace

→

SUBAGENTE

implementador

Sigue la skill: lee tarea, diseño y criterios. Programa solo esa tarea

→

SUBAGENTE

revisor-spec

Solo lectura. Cumple o no cumple, criterio a criterio

→

PERSONA

Decide

Corrige el código, o cambia antes la spec

Mientras tanto, las rules se cargan solas según el archivo que se toca.

Mismo método, distinta herramienta

	En clase, con Copilot	En la práctica, con Cursor
Cómo se generan	Archivo a archivo, revisando cada uno	Tarea a tarea, con un comando
Quién exige seguir la spec	La persona, en cada petición	La configuración del repositorio
Quién revisa	La persona	Un subagente y la persona
Contexto entre sesiones	Lo aporta la persona	Archivos que Cursor carga solo

La disciplina pasa de la persona a archivos del repositorio.

Ventajas e inconvenientes

Ventajas

- Todo el equipo comparte las mismas instrucciones, en Git
- Specs y configuración se revisan con un diff, como el código
- El comando repite siempre la misma cadena de trabajo
- Quien construye no es quien comprueba
- Cada sesión carga solo las rules que necesita

Inconvenientes

- Nada obliga a usar el comando: depende del equipo
- Las rules orientan a la IA, pero no la bloquean
- Un revisor IA no sustituye a los tests
- Escribir y mantener las specs lleva tiempo

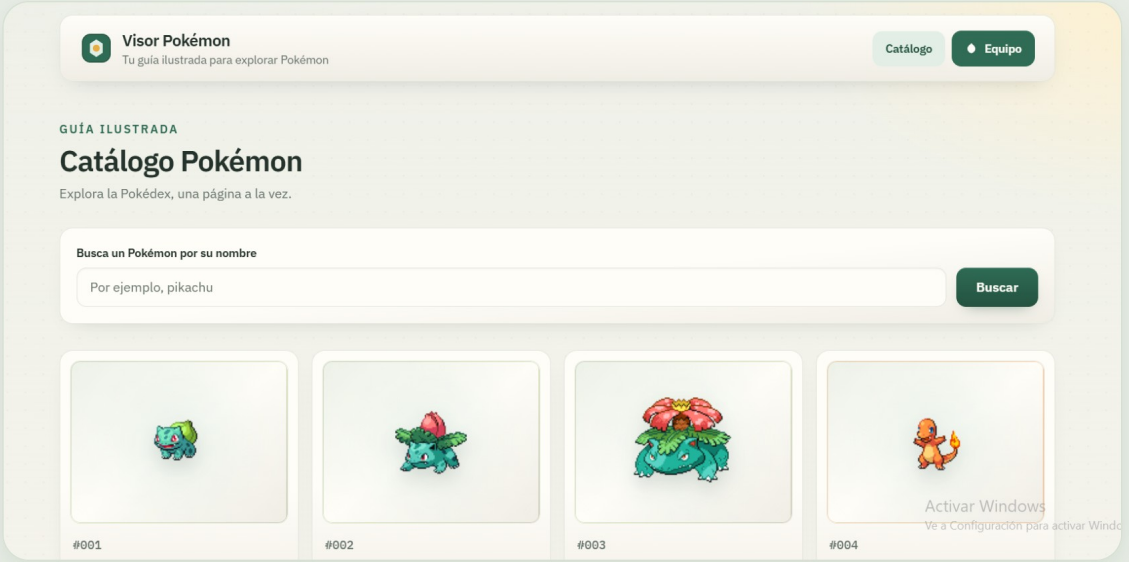
PARTE B · LA APLICACIÓN

Visor Pokémon

Catálogo de Pokémon sobre PokeAPI. Sin cuentas ni servidor propio.

- **Catálogo** paginado, de 20 en 20
- **Búsqueda** con sugerencias
- **Ficha** con tipos, altura, peso y estadísticas
- **Equipo** de hasta seis, con cobertura de tipos

React + TypeScript + Vite + Tailwind



`sergiibut05.github.io/Cursor_SDD_Proyect`

Cómo aplicamos el método

1 · 23 OCT

Specs y configuración de Cursor, antes del código

2 · 23 OCT

Cambio de estilo en la spec, no en el código

3 · 25 OCT

Código de T1 a T7 y README

4 · 26 OCT

T8 Pages · **T9** cambio C1 · **T10** tarjetas holo

VERIFICACIÓN DEL RESULTADO

35/35
tests automáticos pasan

12/12
textos exactos de los requisitos

0
errores de compilación

Dificultades y soluciones

Dificultad	Cómo se resolvió
Un efecto visual nuevo chocaba con la rule de aspecto	Primero se escribió una excepción acotada en la rule y en el diseño; después, el código
PokeAPI solo busca por el nombre en inglés	Un índice de nombres en español, cargado una vez por sesión
La implementación propuso mejoras que no estaban en la spec	Desde la T8, cada tarea nueva entra primero en la spec

Cada problema se resolvió en la spec antes que en el código.

El cambio de requisitos: C1

ANTES

Búsqueda solo por el nombre exacto en inglés. El equipo solo informa de las debilidades.

DESPUÉS

Sugerencias por prefijo, también en español. Cada debilidad filtra el catálogo con los Pokémon que la cubren.

1 · SPEC

Requisitos, diseño, rule pokeapi y nueva tarea T9

→

2 · CÓDIGO

Cliente de PokeAPI, catálogo y equipo

→

3 · VERIFICACIÓN

Tests nuevos: 26/26 pasan

Valoración crítica

Lo que funcionó

- Con specs concretas, la IA cumplió los criterios
- Criterios exactos se convierten en tests: 35
- Hasta los cambios visuales pasaron por la spec

Lo que mejoraríamos

- Guardar registro del uso del agente: bitácora y capturas
- Requisitos de accesibilidad y móvil, no solo en el diseño
- Probar un hook que avise si el código se adelanta a la spec

La IA cumple lo que está escrito. La calidad depende de cómo escribimos la spec.

Gracias.
¿Preguntas?

github.com/Sergiibut05/Cursor_SDD_Proyect